LAB Assignment: Branch Prediction Exploration and Implementation using ChampSim

Computer Architecture — Branch Prediction & Microarchitectural Simulation

Objective

The objective of this assignment is to familiarize students with **ChampSim**, understand existing branch prediction mechanisms, evaluate their performance, and implement progressively more sophisticated branch predictors.

Students will work with ChampSim to:

- Explore existing branch predictors.
- Understand and compare branch-prediction performance.
- Implement a **2-bit saturating counter predictor**.
- Design and evaluate a **table-based branch predictor** with different table sizes.

Step 1 — Explore Existing Branch Predictors

Task

Explore the branch predictors already available in the provided ChampSim implementation.

Students should:

1. Identify the branch predictors available in ChampSim.
2. Read and understand the implementation of at least **three different predictors**.
3. Identify:
    - Predictor state
    - Number of entries
    - Indexing mechanism
    - Update mechanism
    - Prediction mechanism
4. Explain how the predictors differ from each other.

Suggested predictors

Students may explore predictors such as:

- Bimodal
- GShare
- Perceptron
- Any other predictor available in the provided ChampSim version

Deliverable

Prepare a short table:

Predictor	Type	Table/State	Indexing	Update Mechanism
Bimodal				
GShare				
Predictor 3				

Students should also provide a **brief explanation of each predictor**.

Step 2 — Compare Existing Predictors

Task

Run the selected branch predictors using the **same traces and simulation configuration**.

For each predictor, collect and compare:

Required metrics

Processor performance

- IPC
- Instructions per cycle / simulation cycles

Branch prediction

- Total branches
- Correct predictions
- Mispredictions
- Branch prediction accuracy
- Misprediction rate

Cache-related metrics

- L1I hit/miss rate
- L1D hit/miss rate
- LLC hit/miss rate

Students should use the **same configuration, warmup period, simulation period, and traces** for all predictors.

Suggested results table

Predictor	IPC	Branch Accuracy	Misprediction Rate	L1I Miss Rate	LLC Miss Rate
Bimodal					
GShare					
Perceptron					

Analysis

Students should answer:

1. How does branch prediction accuracy vary between predictors?
2. Does higher branch prediction accuracy always result in higher IPC?
3. How does branch misprediction affect processor performance?
4. Do different predictors affect cache-related statistics?
5. Which characteristics of the workload influence predictor performance?

> Students should **justify observations using the collected statistics**, rather than simply stating which predictor performed better.

Step 3 — Implement a 2-Bit Branch Predictor

Task

Implement a **2-bit saturating counter branch predictor** in ChampSim.

Each predictor entry should contain a 2-bit counter with four states:

00 → Strongly Not Taken
01 → Weakly Not Taken
10 → Weakly Taken
11 → Strongly Taken

Prediction rule

00 → Not Taken
01 → Not Taken
10 → Taken
11 → Taken

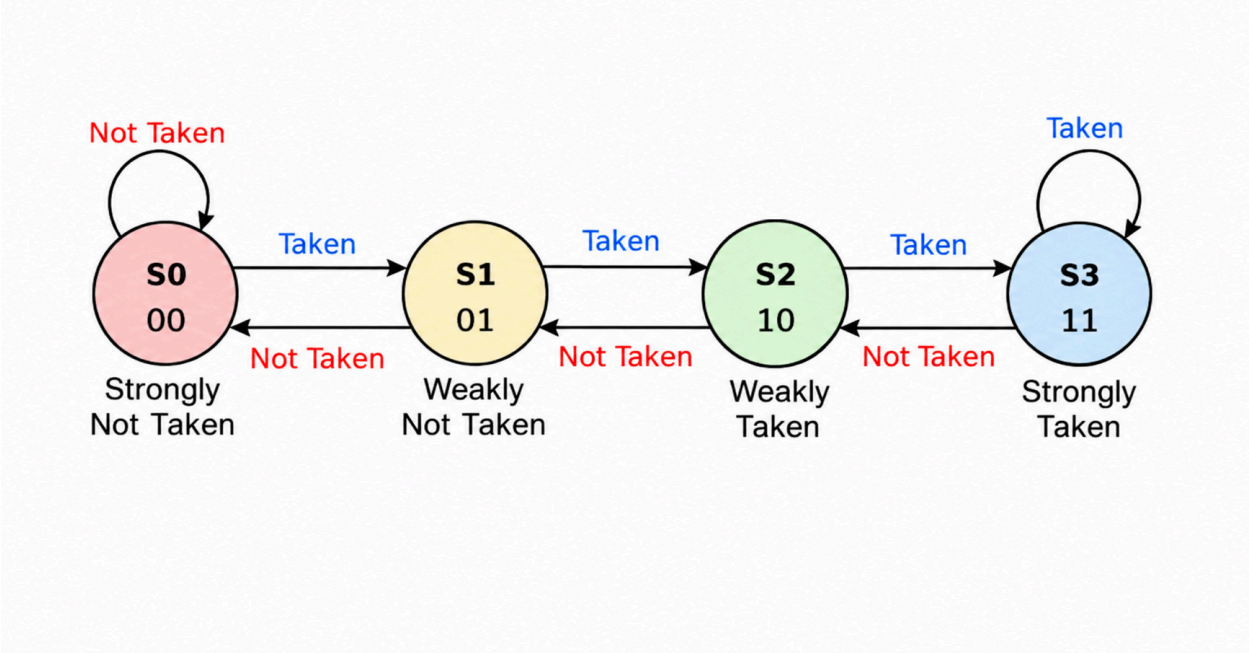
Update rule

For a **Taken** branch:

00 → 01
01 → 10
10 → 11
11 → 11

For a **Not-Taken** branch:

11 → 10
10 → 01
01 → 00
00 → 00



The predictor should use a table indexed using appropriate branch information such as the **branch PC**.

Required experiments

Run your implementation using at least **three traces**.

Compare your predictor against one existing predictor from Step 1.

Results

Report:

Predictor	IPC	Accuracy	Mispredictions	Misprediction Rate
Existing predictor				
Your 2-bit predictor				

Students should explain

- How the predictor is indexed.
- Why two bits are used instead of one.
- How the four states work.
- How the predictor handles repeated Taken/Not-Taken patterns.
- Cases where the predictor performs poorly.

Step 4 — Table-Based Branch Predictor

Now extend the idea from Step 3.

Task

Create a **table-based branch predictor** whose prediction table contains multiple 2-bit counters.

Students must evaluate the effect of changing the number of predictor entries.

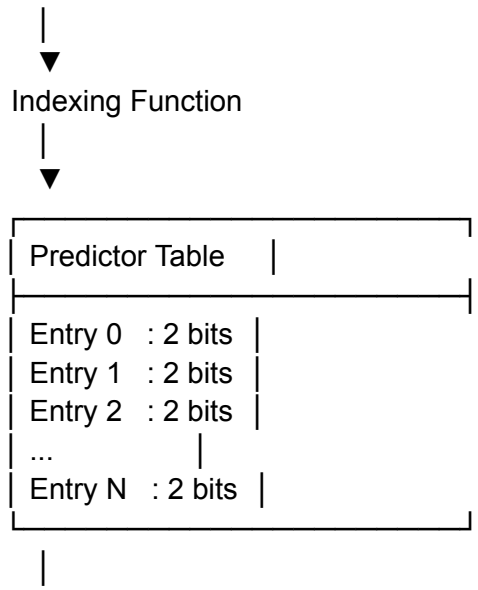
Required table sizes

Students must evaluate:

1K entries
2K entries
4K entries
8K entries
16K entries

Each entry should contain a **2-bit saturating counter**.

For example:



▼
Prediction

Required experiments

Run the predictor with:

- 1K entries
- 2K entries
- 4K entries
- 8K entries
- 16K entries

Keep **all other simulation parameters identical**.

Required results

Table Size	IPC	Branch Accuracy	Mispredictions	Misprediction Rate
1K				
2K				
4K				
8K				
16K				

Final Submission

Students should submit:

1. Source Code

Modified ChampSim branch-predictor implementation.

2. Experimental Results

A CSV/Excel file containing all experimental results.